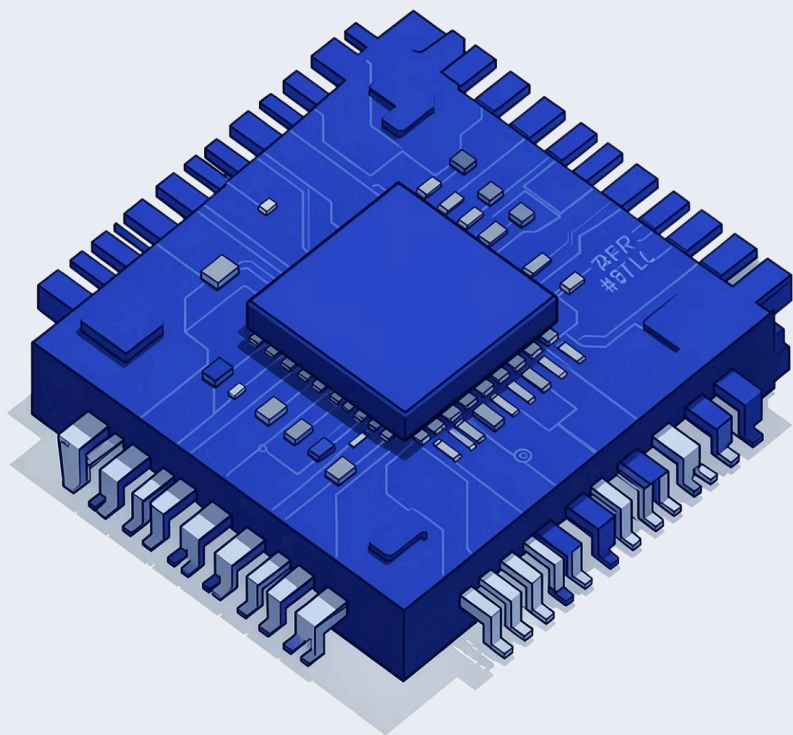




O Processador: O Cérebro do Computador

Uma jornada pelas unidades funcionais que fazem o computador pensar — do Clock ao Pipeline, da Memória Cache aos Periféricos.

AULAS 8 E 9

ARQUITETURA DE COMPUTADORES

Prof. MSc. Cloves Rocha

A Missão do Processador: Executar Instruções

O processador — também chamado de **CPU (Central Processing Unit)** — é o componente central responsável por toda a inteligência computacional de um sistema. Sua existência tem um único e fundamental propósito: executar instruções de forma rápida, precisa e ininterrupta.

Buscar

A CPU localiza a próxima instrução na memória de instruções, guiada pelo Contador de Programa (PC), que indica o endereço exato de onde buscar.

Decodificar

A instrução binária é interpretada pelo decodificador: quais operandos usar, qual operação realizar e quais unidades funcionais acionar.

Executar

A ULA ou outra unidade especializada realiza a operação — aritmética, lógica ou de controle — e grava o resultado no banco de registradores ou na memória.

Além do ciclo busca-decodifica-executa, o processador também **comanda todos os outros componentes** do sistema: memória, dispositivos de entrada/saída, barramentos e co-processadores. Sem a CPU, nenhuma parte do computador sabe o que fazer. É ela quem transforma dados brutos em informação útil, e instrução em ação.

O Ritmo da Execução: O Clock

O Clock é o "**maestro**" que dita o ritmo de todo o processamento interno da CPU. Ele gera pulsos elétricos periódicos — os chamados **ciclos de clock** — que sincronizam todas as operações do processador, garantindo que cada unidade funcional trabalhe em harmonia e no momento certo.

A **frequência do clock**, medida em Hertz (Hz), determina quantos ciclos são executados por segundo. Um processador moderno de **3.5 GHz** realiza impressionantes 3,5 bilhões de ciclos por segundo.

Conceitos Fundamentais do Clock

Período do Clock

Tempo entre dois pulsos consecutivos (ex: ~0,29 ns para 3,5 GHz). Quanto menor o período, mais rápido o processador.

Frequência vs. Desempenho

Maior frequência geralmente significa maior desempenho, mas também maior consumo de energia e geração de calor — um equilíbrio crítico no design moderno.

CPI (Ciclos por Instrução)

Nem toda instrução completa em 1 ciclo. Operações complexas como divisão de ponto flutuante podem exigir múltiplos ciclos, enquanto o pipeline busca reduzir o CPI médio.

Clock em Sistemas Modernos

Processadores multicore ajustam dinamicamente a frequência via *Turbo Boost* e gerenciamento de energia, otimizando clock por núcleo conforme a carga de trabalho.

Unidades Funcionais do Processador

Cada componente interno do processador tem uma função específica e essencial. Juntos, eles formam o **caminho de dados** — a infraestrutura que permite a execução de qualquer programa.



PC — Contador de Programa

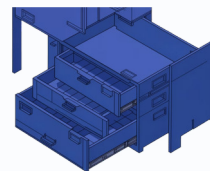
Registrador especial que armazena o **endereço da próxima instrução** a ser buscada na memória.

Atualizado automaticamente após cada instrução (geralmente $PC = PC + 4$ para instruções de 32 bits). Em desvios condicionais, o PC pode ser carregado com um endereço diferente, alterando o fluxo de execução.



Memória de Instruções

Região de memória (ou cache dedicada) onde as **instruções do programa** estão armazenadas em formato binário. O processador acessa esta memória a cada ciclo de busca. Uma instrução típica RISC ocupa 32 bits e codifica opcode, registradores fonte e destino, e imediatos.



Banco de Registradores

Conjunto de **registradores de uso geral** de altíssima velocidade, localizados dentro da própria CPU. Processadores RISC-V possuem 32 registradores de 32 ou 64 bits. Permitem acesso em um único ciclo de clock, eliminando a latência da memória principal durante os cálculos.



ULA — Unidade Lógica e Aritmética

O "**motor computacional**" da CPU. Executa adição, subtração, AND, OR, XOR, comparações e deslocamentos de bits. Recebe dois operandos do banco de registradores e produz um resultado, além de flags de status (zero, overflow, carry) usadas em desvios condicionais.

Memória de Dados e o Caminho de Dados Completo

A **Memória de Dados** é o componente onde o processador lê (*load*) e escreve (*store*) dados durante a execução de um programa. Diferente da memória de instruções — acessada apenas para leitura — a memória de dados suporta tanto leituras quanto escritas, sendo fundamental para qualquer operação que manipule variáveis, vetores ou estruturas de dados.

Operações de Acesso à Memória de Dados

LOAD (lw)

O processador calcula um endereço efetivo (base + offset), acessa a memória de dados nesse endereço e carrega o valor lido em um registrador destino. Ex: `lw x5, 8(x2)`

STORE (sw)

O valor de um registrador é escrito na memória de dados no endereço calculado. Ex: `sw x5, 12(x2)` — grava o conteúdo de x5 no endereço `x2 + 12`.

Construção do Caminho de Dados

O **caminho de dados (datapath)** é a infraestrutura de hardware que conecta todas as unidades funcionais. Sua construção envolve:

- Multiplexadores (MUX) para seleção entre múltiplas fontes de dados
- Somadores dedicados para cálculo de endereços e atualização do PC
- Extensores de sinal para imediatos de 12 ou 20 bits
- Barramentos de dados interligando registradores, ULA e memória
- Sinais de controle gerados pela unidade de controle para configurar cada MUX e habilitar escritas

 O caminho de dados determina diretamente quais instruções o processador pode executar e em quantos ciclos cada uma requer.

Busca, Decodificação e Execução de Instruções

O ciclo de execução do processador é um processo rigorosamente estruturado. Cada instrução percorre um caminho bem definido desde sua localização na memória até o momento em que seus efeitos são gravados. Compreender este ciclo é fundamental para entender como o hardware transforma código em computação real.



Determinação do Controle de Hardware

Para cada instrução, a **Unidade de Controle** decodifica o opcode e gera os sinais que configuram o hardware:

- **RegWrite**: habilita escrita no banco de registradores
- **MemRead / MemWrite**: habilita acesso à memória de dados
- **ALUSrc**: seleciona se o segundo operando da ULA vem de um registrador ou de um imediato
- **MemToReg**: seleciona se o dado gravado no registrador vem da ULA ou da memória
- **Branch**: habilita atualização condicional do PC

Exemplo: Instrução ADD no RISC-V

01

Busca

PC = 0x0010; instrução `add x1, x2, x3` é lida da memória; $PC \leftarrow PC + 4$

02

Decodificação

Opcode identifica instrução tipo-R; registradores x2 e x3 são lidos do banco

03

Execução

ULA recebe valores de x2 e x3; realiza adição; produz resultado e flag Zero

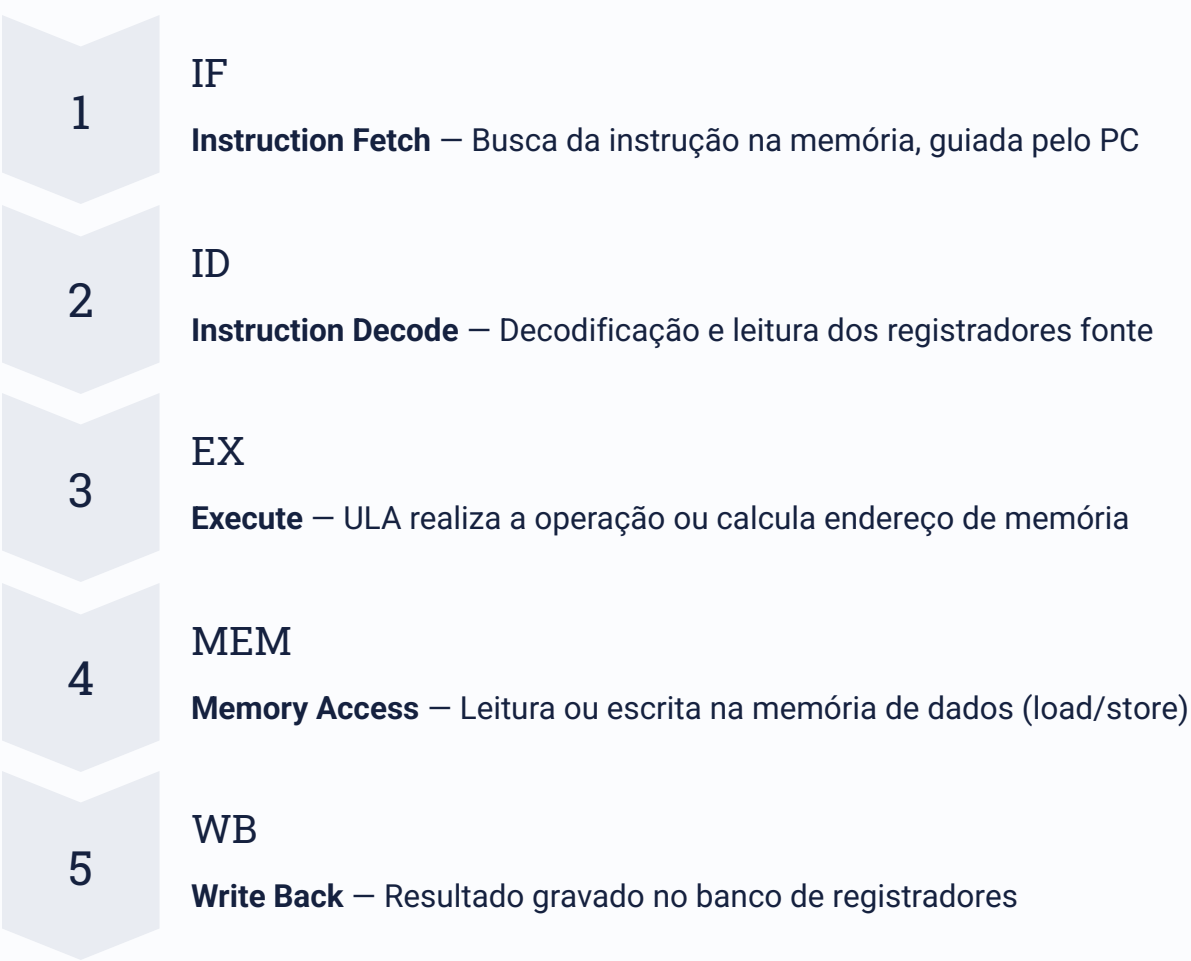
04

Escrita

Resultado da ULA é escrito em x1 no banco de registradores; ciclo encerrado

Pipeline: Paralelismo no Coração do Processador

O **Pipeline** é uma das técnicas mais poderosas da arquitetura de computadores. A ideia é simples e elegante: assim como uma linha de montagem industrial processa vários carros simultaneamente em etapas diferentes, o processador pipeline executa múltiplas instruções ao mesmo tempo, cada uma em uma etapa diferente do ciclo.



Ganho de Desempenho com Pipeline

Sem pipeline, cada instrução ocupa todos os estágios sequencialmente. Com pipeline de 5 estágios, em regime estacionário, **5 instruções diferentes** estão em execução simultânea — uma em cada estágio. O throughput ideal é de **1 instrução por ciclo** (IPC = 1), contra 5 ciclos por instrução sem pipeline. O **caminho de dados em pipeline** adiciona registradores de pipeline entre os estágios (IF/ID, ID/EX, EX/MEM, MEM/WB) para isolar e propagar os dados e sinais de controle corretamente.

Conflitos (Hazards) no Pipeline

Conflito de Dados

Instrução depende do resultado de uma instrução ainda no pipeline. Soluções: **forwarding** (encaminhamento direto do resultado) ou **stall** (inserção de bolhas).

Conflito em Desvios

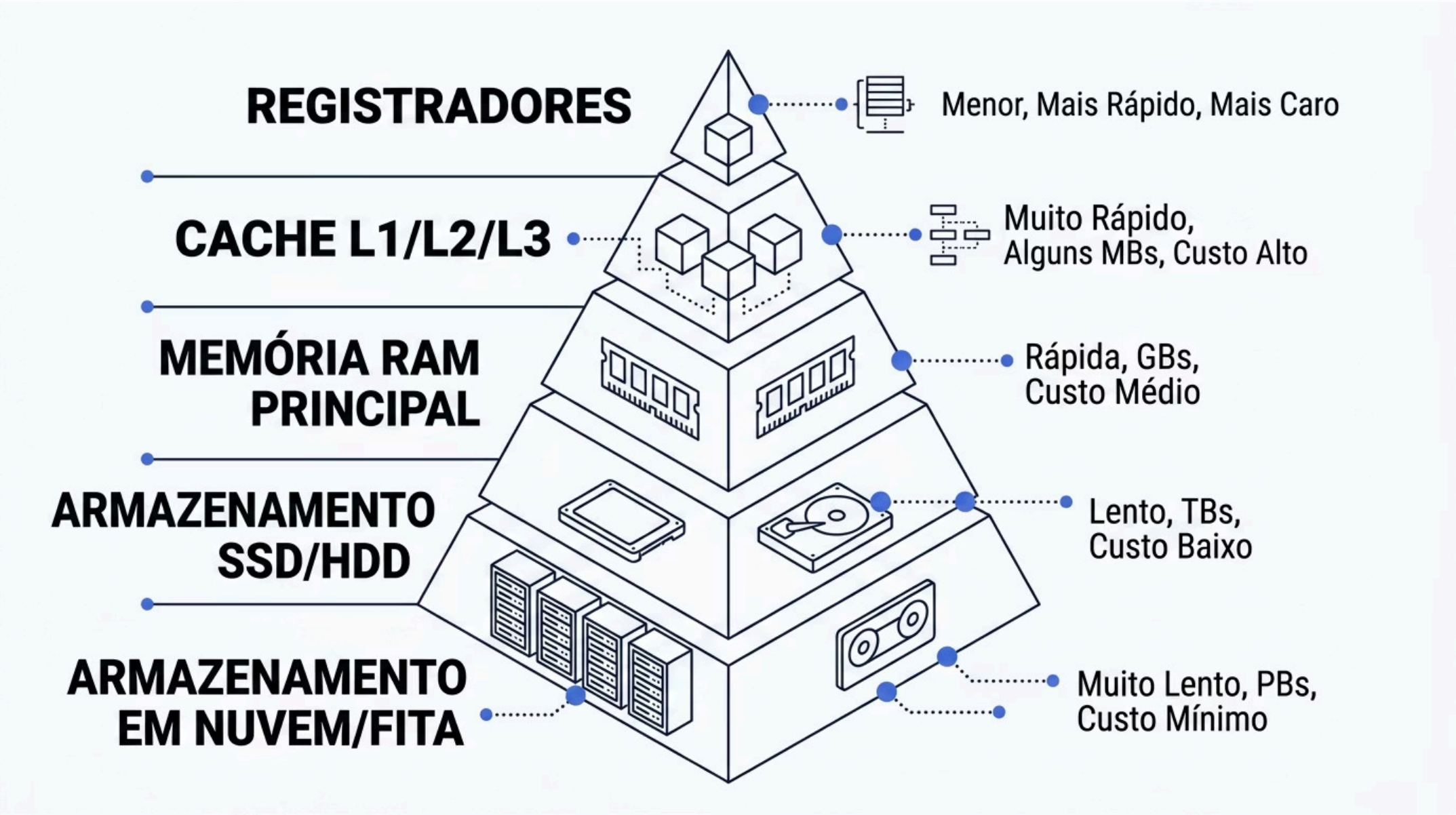
Desvios condicionais alteram o PC, mas o pipeline já buscou as próximas instruções. Soluções: **branch prediction**, **branch delay slot** ou flush do pipeline.

Conflito Estrutural

Duas instruções tentam usar o mesmo recurso de hardware simultaneamente. Resolvido com memórias separadas para instruções e dados.

Hierarquia de Memória: Do Cache à Memória Virtual

A **Hierarquia de Memória** é a solução arquitetural para o grande desafio da computação moderna: o processador opera muito mais rápido do que a memória principal. A solução é organizar a memória em camadas, cada uma com características diferentes de velocidade, tamanho e custo.



Memória Cache: O Acelerador Invisível

A **cache** é uma memória pequena e ultrarrápida localizada dentro ou próxima ao processador. Ela guarda cópias das regiões de memória mais acessadas, explorando dois princípios fundamentais:

- **Localidade Temporal**: dados acessados recentemente tendem a ser acessados novamente em breve
- **Localidade Espacial**: dados próximos ao endereço acessado tendem a ser acessados logo depois

A **associatividade da cache** define como os blocos de memória são mapeados para as linhas de cache: mapeamento direto (1-way), N-way set associative ou totalmente associativo — cada abordagem equilibra velocidade de acesso com taxa de miss.

Memória Virtual

A **memória virtual** é uma abstração que permite que programas usem mais memória do que a fisicamente disponível. O sistema operacional, com suporte do hardware (MMU — Memory Management Unit), mantém uma tabela de páginas que traduz **endereços virtuais** (usados pelos programas) em **endereços físicos** (localizações reais na RAM).

- Páginas não utilizadas são armazenadas no disco (**swap**)
- Um **page fault** ocorre quando uma página virtual não está na RAM
- A **TLB (Translation Lookaside Buffer)** é uma cache especial para traduções de endereços, acelerando o processo de mapeamento
- Permite isolamento entre processos e proteção de memória

Acessando Periféricos: I/O e Conexão ao Sistema

Os **dispositivos de Entrada e Saída (I/O)** são a ponte entre o computador e o mundo externo. Teclados, monitores, discos, placas de rede — todos precisam se comunicar com o processador e a memória de forma eficiente. A arquitetura de I/O define como essa comunicação acontece.



Tipos de Dispositivos de I/O

Dispositivos variam amplamente em **velocidade**, **direção e natureza do dado**. Classificam-se em: dispositivos de *bloco* (HDD, SSD — transferem blocos de dados), dispositivos de *caractere* (teclado, mouse — transferem bytes individuais) e dispositivos de *rede*. Cada tipo exige estratégias de acesso distintas.



Conexão ao Processador e à Memória

Dispositivos se conectam via **barramentos** (PCI Express, USB, SATA). O controlador de I/O atua como intermediário, traduzindo protocolos e gerenciando transferências. O **DMA (Direct Memory Access)** permite que dispositivos transfiram dados diretamente para a RAM sem ocupar o processador a cada byte.



Interface com a Memória e o S.O.

A **I/O mapeada em memória** reserva endereços do espaço de memória para registradores de controle dos dispositivos. O processador interage com periféricos usando instruções comuns de load/store. O **Sistema Operacional** gerencia o acesso via *drivers*, garantindo que múltiplos processos compartilhem dispositivos com segurança e eficiência através de interrupções e chamadas de sistema.

 Interrupções são o mecanismo fundamental pelo qual dispositivos notificam o processador que precisam de atenção — permitindo que a CPU execute outras tarefas enquanto aguarda operações lentas de I/O completarem.

Síntese: O Processador e a Arquitetura Completa

Ao longo destas aulas, exploramos como cada componente do processador e do sistema computacional se integra para executar programas com eficiência. Cada conceito estudado resolve um problema real de desempenho ou funcionalidade.

Unidades Funcionais

Clock, PC, Memória de Instruções, Banco de Registradores, ULA e Memória de Dados formam o núcleo do caminho de dados — cada um com papel insubstituível na execução de instruções.

Pipeline

O pipeline de 5 estágios (IF, ID, EX, MEM, WB) multiplica o throughput do processador, com forwarding e branch prediction mitigando os conflitos inevitáveis.

Hierarquia de Memória

Cache (com localidade e associatividade), RAM e memória virtual formam uma hierarquia que equilibra velocidade, capacidade e custo, mantendo o processador alimentado com dados.

Periféricos e I/O

DMA, barramentos, I/O mapeada em memória e interrupções conectam o processador ao mundo real, com o S.O. gerenciando o acesso seguro e eficiente a todos os dispositivos.

"A arquitetura de computadores é a arte de equilibrar velocidade, custo, energia e complexidade para que bilhões de instruções simples se transformem em experiências extraordinárias."

5

Estágios de Pipeline

IF → ID → EX → MEM → WB

32

Registradores RISC-V

Banco de registradores de uso geral

3.5GHz

Clock Moderno

3,5 bilhões de ciclos por segundo

5

Níveis de Memória

Registradores → Cache → RAM → SSD → Nuvem